

Google Docs

Collaborative editor – "మార్పులని server కి పంపి, అందరికీ ప్రసారం చేద్దాం" అని అందరూ మొదలుపెడతారు. అదే సరైన ఆలోచన, మరియు దాన్ని నేరుగా రాస్తే **ఒక్క session కూడా** సరిగ్గా ముగియదు.

మూడు విరుపులు. మొదటిది – పూర్తి document ని save చేస్తే **N మందిలో ఒక్కరి పని** మాత్రమే మిగులుతుంది, మరియు **తరచుగా save చేయడం సహాయపడదు**. రెండోది – స్థానాలతో ops పంపితే **60 to 0 sessions** ఒకేలా ముగిశాయి. మూడోది – history **document కోట్ 134 రెట్లు**, మరియు దాన్ని కత్తిరించడమే మీ offline కిటికీని నిర్ణయిస్తుంది.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసినదే.

ఈ **PROBLEM** మిగతా **HLD PROBLEMS** కంటే ఎందుకు వేరు

మిగతా వాటిలో మీరు **పెద్దదనం** తో పోరాడతారు – ఎక్కువ requests, ఎక్కువ data, ఎక్కువ machines.
పరిష్కారాలు sharding, caching, replication.

ఇక్కడ అవేవీ సహాయపడవు, ఎందుకంటే **ఒక document ని shard చేయలేరు**. ఒక document ని సవరిస్తున్న అందరూ **ఒక చోటికి** రావాలి – లేకపోతే ఎవరి అక్షరం ఎక్కడికి వెళ్తుందో నిర్ణయించేవాళ్ళు ఉండరు.

కాబట్టి ఈ problem **ఒక algorithm problem**, ఒక infrastructure problem కాదు. మరియు ఆ algorithm – operational transform – **మీరు అనుకున్నదానికంటే చాలా సున్నితమైనది**. §13 lo చూపిస్తాను: transform lo ఒక `<` ని `<=` గా మార్చితే **200 lo 184 sessions** విరుగుతాయి.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఏది సమస్య కాదు
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: document ని save చేయడం

3. Step – ప్రతి కొన్ని సెకన్లకీ save
4. **మొదటి విరుపు** – N మందిలో ఒక్కరి పని మిగులుతుంది
5. Step – document కాదు, మార్పులు పంపడం

Part 3 – రెండో విరుపు: స్థానాలు

6. Step – `insert(pos, ch)` ని ప్రసారం చేయడం
7. **రెండో విరుపు** – 60 lo 0 sessions ఒకేలా ముగిశాయి
8. Step – transform · ఆరు సందర్భాలు
9. Step – ఒకేసారి ఒక్క op · మరియు దాని ఖరీదు

Part 4 – మూడో విరుపు: చరిత్ర

10. Step – server log ని ఉంచాలి
11. **మూడో విరుపు** – log, document కంటే 134 రెట్లు
12. Step – snapshot + కత్తిరించిన log · offline కీటికీ

Part 5 – నిరూపణ మరియు ప్రదర్శన

13. మొత్తం code · 1 లక్ష ops · mutation testing

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
15. నోటీతో చెప్పాల్సిన English script
16. Follow-ups – cursors, undo, offline, CRDT
17. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఏది సమస్య కాదు

10 బిలియన్ documents. రోజుకి 100 కోట్ల వాడుకరులు. ఒక document ని చాలామంది ఒకే సమయంలో సవరించగలగాలి, మరియు ప్రతి ఒక్కరికీ వెంటనే కనిపించాలి.

మొదట మొత్తం load ని కొలుద్దాం:

10 బిలియన్ docs • రోజుకి 100 కోట్ల వాడుకరులు

ఏకకాలంలో edit చేస్తున్నవారు : 3,00,00,000

ఒక్కొక్కరు నిమిషానికి : 200 అక్షరాలు

→ సెకనుకి మొత్తం : 10,00,00,000 keystrokes

ఒక్కో op 100 bytes : 10.0 GB/s

ఒక machine సెకనుకి 50,000 ops → 2,000 machines

2,000 machines. పెద్ద సంఖ్య, కానీ అసాధ్యమైనది కాదు – మరియు ఇది పూర్తిగా shard చేయదగినది, ఎందుకంటే రెండు వేర్వేరు documents ఒకదానితో ఒకటి మాట్లాడవు.

కాబట్టి ముఖ్యమైన సంఖ్య అది కాదు. ఇది:

ఒక doc lo	1 మంది	→ ఒక్కో keystroke	0 మందికి	→ సెకనుకి	0 సందేశాలు	ఆ ఒక్క doc కి
ఒక doc lo	2 మంది	→ ఒక్కో keystroke	1 మందికి	→ సెకనుకి	7 సందేశాలు	ఆ ఒక్క doc కి
ఒక doc lo	5 మంది	→ ఒక్కో keystroke	4 మందికి	→ సెకనుకి	67 సందేశాలు	ఆ ఒక్క doc కి
ఒక doc lo	10 మంది	→ ఒక్కో keystroke	9 మందికి	→ సెకనుకి	300 సందేశాలు	ఆ ఒక్క doc కి
ఒక doc lo	50 మంది	→ ఒక్కో keystroke	49 మందికి	→ సెకనుకి	8167 సందేశాలు	ఆ ఒక్క doc కి

ఈ **PROBLEM** యొక్క ఒకే ఒక నిర్మాణాత్మక నిజం

Documents మధ్య shard చేయడం చాలా సులభం. ఒక document లోపల shard చేయడం అసాధ్యం.

ఎందుకంటే ఒక document యొక్క సరైన స్థితిని నిర్ణయించాలంటే – ఎవరి అక్షరం ఎక్కడికి వెళ్ళాలో – **ఒక చోట ఒక క్రమం** ఉండాలి. ఆ క్రమాన్ని రెండు machines మధ్య పంచలేరు.

కాబట్టి ఒక document = **ఒక process**, మరియు ఆ process ని పెద్దది చేయడమే మే ఎక్సైకి scaling దారి.
అదృష్టవశాత్తూ ఒక doc lo 50 మంది కంటే ఎక్కువ ఉండరు – సెకనుకి 8,167 సందేశాలు, ఒక్క process కి పెద్ద భారం కాదు.

మిగతా మొత్తం doc ఈ ఒక్క process లోపల ఏమి జరగాలో గురించి.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	ఎందుకు అడుగుతున్నాం
ఒక doc lo ఎంతమంది ఏకకాలంలో?	50 అనేది ఒక process; 5,000 అయితే వేరే problem (§16)
రెండు మార్పులు ఢీకొంటే ఎవరు గెలవాలి?	ఇదే §8. "ఇద్దరూ గెలవాలి" అనేదే మొత్తం సవాలు
Offline edit చేయగలగాలా?	అవును అంటే §12 – మీ log పరిమాణమే మీ offline కిటికీ
చరిత్ర కావాలా – ఎవరు ఏమి రాశారు?	అవును అంటే §11 – log ని కత్తిరించలేరు, మరియు అది 134 రెట్లు
ఎంత ఆలస్యం ఆమోదయోగ్యం?	§9 – round-trip time నేరుగా మీ buffer లోతుని నిర్ణయిస్తుంది
Rich text (bold, పట్టికలు, బొమ్మలు) కావాలా?	transform సందర్భాలు ఆరు నుంచి చాలా పెరుగుతాయి – §16
Undo కావాలా?	అనుకున్నదానికంటే చాలా కష్టం – §16
Server చనిపోతే ఏమవుతుంది?	Log durable గా ఉండాలి; లేకపోతే ఆమోదించిన ops పోతాయి

ఒక ప్రశ్న అడిగితే మరు దీన్ని నిజంగా రాశారని తెలుస్తుంది

"వాడుకరి కొట్టిన అక్షరం వెంటనే కనిపించాలా, లేక server ఆమోదం వరకు ఆగిపో?"

ఎందుకంటే "వెంటనే" అనే ఒక్క మాటే **మొత్తం సంక్లిష్టతని సృష్టిస్తుంది**. Server ఆమోదం వరకు ఆగితే ఈ problem కి transform అవసరం లేదు – ఒక queue సరిపోతుంది, మరియు 200 ms ఆలస్యంతో టైపింగ్ భరించలేనిదిగా ఉంటుంది.

"వెంటనే" అంటే **client server ని నమ్మకుండా ముందుకు వెళ్తుంది**, ఆపై ఇద్దరూ వేరే స్థితిలో ఉంటారు, ఆపై వాటిని కలపాలి. §8 మొత్తం ఆ కలపడం గురించే.

Part 2 – మొదటి విరుపు: document ని save చేయడం

3. Step – ప్రతి కొన్ని సెకన్లకీ save

అత్యంత సహజమైన మొదలు – మీరు ఇప్పటికే ఒక text editor రాశారనుకోండి, దానికి "auto-save" జోడిస్తున్నారు:

```
setInterval(() => server.save(docId, editor.getText()), 2000); // ప్రతి 2 సెకన్లకీ  
setInterval(() => editor.setText(server.load(docId)), 5000); // ప్రతి 5 సెకన్లకీ
```

Server వైపు ఇంకా సులభం: `docs[id] = text`. చివరిగా వచ్చినది నిలుస్తుంది – **last write wins**.

ఒక్క వాడుకరికి ఇది సరిగ్గా పనిచేస్తుంది. ఇద్దరితో?

4. మొదటి విరుపు – N మందిలో ఒక్కరి పని మిగులుతుంది

2 నిమిషాల session, ఒక్కొక్కరు నిమిషానికి 200 అక్షరాలు, 40 seeds:

పూర్తి document ని save చేస్తే (last-write-wins) • 2 నిమిషాల session

వాడుకరులు	కొట్టిన అక్షరాలు	మిగిలినవి	పోయినవి	నష్టం %	doc "గెలికె" సార్లు
1	16,139	16,139	0	0.0%	0
2	32,123	15,925	16,198	50.4%	480
3	48,177	16,039	32,138	66.7%	960
5	80,126	16,000	64,126	80.0%	1,920
10	1,60,049	15,924	1,44,125	90.1%	4,320

"మిగిలినవి" అనే నిలువు వరుసని చూడండి – అది మారడం లేదు

1 వాడుకరి: 16,139 మిగిలాయి. 10 వాడుకరులు: 15,924 మిగిలాయి.

పది రెట్లు ఎక్కువమంది పనిచేశారు, మరియు **document lo అంతే పని ఉంది**. నష్టం సరిగ్గా $(N-1)/N = 50.4\%$, 66.7% , 80.0% , 90.1% .

అంటే ఇది "కొంత పని పోయింది" కాదు. **ఎప్పుడూ సరిగ్గా ఒక్కరి పని మాత్రమే మిగులుతుంది**. మిగతా అందరూ టైప్ చేసినది ఒక భ్రమ – తర్వాతి save దాన్ని తుడిచేస్తుంది.

సహజమైన మొదటి స్పందన: "ఇంకా తరచుగా save చేద్దాం."

ఇంకా తరచుగా save చేస్తే సరిపోతుందా? (3 వాడుకరులు)

save విరామం	నష్టం %
5000 ms	66.7%
2000 ms	66.7%
1000 ms	66.7%
500 ms	66.7%
100 ms	66.7%

5 సెకన్లు → 100 MS • ఒక్క దశాంశం కూడా మారలేదు

ఎందుకంటే తరచుగా save చేయడం నష్టాన్ని తగ్గించదు – త్వరగా జరిగేలా చేస్తుంది. ప్రతి save ఇంకొకరి పనిని తుడుస్తుంది, కాబట్టి ఎక్కువ saves = ఎక్కువ తుడుపులు, ఒక్కొక్కటి చిన్నది.

మొత్తం నష్టం అలాగే ఉంటుంది, ఎందుకంటే అది save వేగం మీద ఆధారపడదు – అది ఏమి పంపుతున్నామో దాని మీద ఆధారపడుతుంది.

5. Step – document కాదు, మార్పులు పంపడం

తప్పు ఎక్కడ జరిగిందో ఇప్పుడు స్పష్టం:

పూర్తి DOCUMENT ని పంపడం అంటే ఒక ఫలితాన్ని పంపడం, ఒక ఉద్దేశాన్ని కాదు

"ఈ document ఇలా ఉండాలి" అని చెబితే – server కి మీరు **ఏమి మార్చారో** తెలియదు. కాబట్టి అది మీ copy ని ఇతరుల copy తో కలపలేదు; అది ఒకదాన్ని ఎన్నుకోవడం తప్పు ఏమీ చేయలేదు.

బదులుగా "నేను 5వ స్థానంలో 'k' చేర్చాను" అని పంపితే – అది ఒక ఉద్దేశం. దాన్ని వేరొకరి ఉద్దేశంతో కలపవచ్చు.

స్థితిని పంచకండి, మార్పులను పంచండి. ఈ ఒక్క వాక్యం lo ఈ problem యొక్క సగం ఉంది.

కాబట్టి ఇప్పుడు ప్రతి keystroke ఒక op:

```
{ type: 'ins', pos: 5, ch: 'k' }  
{ type: 'del', pos: 12 }
```

Client దాన్ని **వెంటనే స్థానికంగా వర్తింపజేస్తుంది** (వాడుకరి ఆగకూడదు), ఆపై server కి పంపుతుంది. Server అందరికీ ప్రసారం చేస్తుంది. అందరూ అన్ని ops ని వర్తింపజేస్తారు.

ఇది సరిగ్గా అనిపిస్తుంది. **మిగతా సగం ఇప్పుడు మొదలవుతుంది.**

Part 3 – రెండో విరుపు: స్థానాలు

6. Step – `insert(pos, ch)` ని ప్రసారం చేయడం

```
function apply(doc, op) {  
  if (op.type === 'ins') doc.splice(op.pos, 0, op.ch);  
  else doc.splice(op.pos, 1);  
}
```

Client: స్థానికంగా వర్తింపజేయి, server కి పంపు. Server: log lo పెట్టి అందరికీ పంపు. వచ్చిన ops ని వర్తింపజేయి.

60 సెకన్ల sessions, 60 seeds, ఒక్కొక్కరు నిమిషానికి 200 అక్షరాలు (20% తొలగింపులు):

7. రెండో విరుపు – 60 lo 0 sessions ఒకేలా ముగిశాయి

ప్రతి keystroke ని ఒక op గా పంపడం • transform లేకుండా • 60 సెకన్ల session

వాడుకరులు	ఆలస్యం	ఒకేలా ముగిసిన sessions	తేడా ఉన్న అక్షరాలు	doc lo %
2	0ms	0/60 (0%)	8.7	4%
2	50ms	0/60 (0%)	69.0	28%
2	200ms	0/60 (0%)	144.7	59%
3	50ms	0/60 (0%)	133.0	37%
3	200ms	0/60 (0%)	240.4	66%
5	200ms	0/60 (0%)	417.5	69%

ఒక్క SESSION కూడా ఒకేలా ముగియలేదు • 200 MS ఆలస్యంతో DOCUMENT LO 59% వేరు

మరియు మొదటి వరుస చూడండి – **0 ms ఆలస్యం** తోనూ 0/60.

అంటే ఇది **network సమస్య కాదు**. ఇద్దరు మనుషులు ఒకే క్షణంలో టైప్ చేస్తే అది ఏకకాలం – network ఎంత వేగంగా ఉన్నా. ఆలస్యం కేవలం **కీటికీని వెడల్పు** చేస్తుంది.

ఒక నిర్దిష్ట session – 2 వాడుకరులు, 3 సెకన్లు:

వాడుకరి A చూస్తున్నది : 9 6 5 10 8 4 0
వాడుకరి B చూస్తున్నది : 9 7 6 3 1 0 2

ఒకే ops, ఒకే సంఖ్య – వేరే documents

క్రమం మాత్రమే కాదు – అక్షరాలే వేరు. A దగ్గర 5, 10, 8, 4 ఉన్నాయి; B దగ్గర 7, 3, 1, 2. ఎందుకంటే `del(pos)` రెండు `replicas` lo **వేరే అక్షరాలను** తొలగించింది.

తప్పు ఎక్కడ జరిగిందో ఒక ఉదాహరణతో:

మొదట ఇద్దరి దగ్గరా: A B C D E

వాడుకరి 1: స్థానం 1 lo 'X' → A X B C D E

వాడుకరి 2: స్థానం 3 lo 'Y' → A B C Y D E (ఎకకాలంలో)

ఇప్పుడు ops మార్చుకుంటారు:

వాడుకరి 1 కి `ins(3, 'Y')` వస్తుంది → A X B Y C D E ← 'Y' C కి *ముందు*

వాడుకరి 2 కి `ins(1, 'X')` వస్తుంది → A X B C Y D E ← 'Y' C కి *తర్వాత*

"స్థానం 3" అనేది ఒక సత్యం కాదు – అది ఒక అభిప్రాయం

వాడుకరి 2 "స్థానం 3" అన్నప్పుడు వారి ఉద్దేశం "C తర్వాత". కానీ ఆ సందేశం వాడుకరి 1 దగ్గరికి చేరేసరికి, ఆ document lo స్థానం 3 **వేరే చోటు** – ఎందుకంటే మధ్యలో 'X' వచ్చింది.

ఒక op ఒక **స్థితిని సూచిస్తూ** ఉంటుంది. ఆ స్థితి మారిపోతే, ఆ op యొక్క అర్థం మారిపోతుంది.

కాబట్టి ఒక op ని వర్తింపజేసే ముందు దాన్ని **ప్రస్తుత స్థితికి అనువదించాలి**. ఆ అనువాదమే **transform**.

8. Step – transform • ఆరు సందర్భాలు

`xf(a, b)` = "b ఇప్పటికే జరిగింది; a ని సర్దుబాటు చేయి". రెండూ **ఒకే స్థితి** నుంచి వచ్చినవి.

కావాల్సిన ఆస్తి (TP1):

```
apply(apply(doc, a), xf(b, a)) === apply(apply(doc, b), xf(a, b))
```

అంటే – **ఎవరి op ముందు వచ్చినా, చివరికి ఒకే document.**

```
function xf(a, b) {
  if (a.type === 'nop' || b.type === 'nop') return a;

  if (a.type === 'ins' && b.type === 'ins') {
    if (a.pos < b.pos) return a; // ముందు ఉంది → మారదు
    if (a.pos > b.pos) return { ...a, pos: a.pos + 1 }; // వెనక ఉంది → ఒకటి
    జరుగు
    return a.site < b.site ? a : { ...a, pos: a.pos + 1 }; // సమానం → site id తో
    tie-break
  }
  if (a.type === 'ins' && b.type === 'del')
    return a.pos <= b.pos ? a : { ...a, pos: a.pos - 1 };
  if (a.type === 'del' && b.type === 'ins')
    return a.pos < b.pos ? a : { ...a, pos: a.pos + 1 };
}
```

```
// del/del
if (a.pos < b.pos) return a;
if (a.pos > b.pos) return { ...a, pos: a.pos - 1 };
return { ...a, type: 'nop' }; // ఇద్దరూ ఒకే అక్షరాన్ని తొలగించారు
}
```

మూడు సూక్ష్మమైన పంక్తులు ఉన్నాయి, మరియు మూడూ **తప్పు రాస్తే fuzz పట్టుకుంటుంది** (§13):

1. INS/INS సమాన స్థానం - ఒక ఏకపక్ష నిర్ణయం, కానీ అందరూ ఒకే నిర్ణయం తీసుకోవాలి

ఇద్దరూ సరిగ్గా ఒకే చోట అక్షరం చేర్చితే - ఎవరిది ముందు? **సరైన జవాబు లేదు**. కానీ ప్రతి replica **ఒకే** జవాబుకి రావాలి, కాబట్టి site id తో నిర్ణయిస్తాం.

దీన్ని తీసేస్తే: **200 lo 191 sessions** విరుగుతాయి (§13).

2. INS/DEL LO <= , DEL/INS LO < - ఎందుకు వేరు?

ins(3) అంటే "3వ స్థానంలోకి రా", del(3) అంటే "3వ అక్షరాన్ని తీసేయ్". స్థానం 3 lo చేర్చడం, స్థానం 3 ని తొలగించడం - ఈ రెండూ **ఒకే చోటుని సూచించవు**.

<= ని < గా మార్చితే **200 lo 192**; < ని <= గా మార్చితే **200 lo 184**.

3. DEL/DEL ఒకే స్థానం → NOP

ఇద్దరూ ఒకే అక్షరాన్ని తొలగిస్తే, రెండేది **ఏమీ చేయకూడదు** - లేకపోతే అది పక్కనున్న అమాయక అక్షరాన్ని తీసేస్తుంది.

దీన్ని తీసేస్తే **200 lo 68** విరుగుతాయి - మిగతా వాటికంటే తక్కువ, ఎందుకంటే ఇద్దరు ఒకే అక్షరాన్ని ఒకేసారి తొలగించడం అరుదు. **అరుదైనది అంటే సురక్షితమైనది కాదు** - అది కేవలం ఆలస్యంగా బయటపడేది.

9. Step - ఒకేసారి ఒక్క op • మరియు దాని ఖరీదు

ఇక్కడ నా మొదటి ప్రయత్నం **500 lo 500 sessions** విఫలమైంది, మరియు కారణం transform కాదు.

నా తప్పు - మరియు అది ఒక నిజమైన DESIGN నియమం

నా client ఒకేసారి **రెండు ops** ని గాలిలోకి పంపింది, రెండింటికీ ఒకే baseRev .

కానీ రెండో op యొక్క ఆధారం "server స్థితి + మొదటి op". Server దాన్ని ఒకే baseRev నుంచి transform చేసినప్పుడు, అది తప్పు ఆధారం మీద పనిచేస్తుంది.

దీనికి పరిష్కారం transform ని బాగుచేయడం కాదు - **ఒకేసారి ఒక్క op మాత్రమే పంపడం**. మిగిలినవి client lo ఒక buffer lo ఆగుతాయి.

```

class Client {
  doc = []; outstanding = null; buffer = []; serverRev = 0;

  local(op) { // వాడుకరి కొట్టినది - వెంటనే కనిపించాలి
    apply(this.doc, op);
    if (this.outstanding) { this.buffer.push(op); return null; }
    this.outstanding = op;
    return { op, baseRev: this.serverRev };
  }

  remote(op, rev) { // వేరొకరి op వచ్చింది
    let r = op;
    if (this.outstanding) {
      const o = this.outstanding;
      this.outstanding = xf(o, r); // నా in-flight op కూడా సర్దుబాటు కావాలి
      r = xf(r, o);
    }
    for (let i = 0; i < this.buffer.length; i++) {
      const b = this.buffer[i];
      this.buffer[i] = xf(b, r);
      r = xf(r, b);
    }
    if (r.type !== 'nop') apply(this.doc, r);
    this.serverRev = rev;
  }

  ack(rev) { // నా op ఆమోదించబడింది → తర్వాతిది పంపు
    this.serverRev = rev;
    this.outstanding = this.buffer.shift() || null;
    return this.outstanding ? { op: this.outstanding, baseRev: rev } : null;
  }
}

```

ఈ నియమం ఉచితం కాదు. **Client సెకనుకి ఒక round-trip కి ఒక op మాత్రమే పంపగలదు.** కొలిచాను:

"ఒకేసారి ఒక్క op" నియమం ఏమి ఖర్చు పెడుతుంది? • 60 సెకన్లు, నిమిషానికి 200 అక్షరాలు

round-trip	సెకనుకి పంపగలిగేవి	buffer సగటు	buffer గరిష్ఠం
20 ms	50 ops/s	0.00	1
50 ms	20 ops/s	0.02	3
100 ms	10 ops/s	0.07	4
200 ms	5 ops/s	0.57	9
500 ms	2 ops/s	37.66	100

200 MS → 500 MS • BUFFER సగటు 0.57 నుంచి 37.66 కి

ఇది క్రమంగా పెరగడం కాదు – ఇది ఒక గోడ. నిమిషానికి 200 అక్షరాలు అంటే సెకనుకి 3.3. Round-trip 500 ms అంటే సెకనుకి 2 ops.

3.3 > 2 – కాబట్టి buffer ఎప్పటికీ ఖాళీ కాదు. అది శాశ్వతంగా పెరుగుతూ ఉంటుంది, మరియు వాడుకరి టైప్ చేసినది ఇతరులకి **నిమిషాల** ఆలస్యంతో కనిపిస్తుంది.

అదే గోడ వేరే వైపు నుంచి – వేగంగా టైప్ చేస్తే:

వేగంగా టైప్ చేస్తే? (round-trip 100 ms)

టైపింగ్ వేగం	సెకనుకి అక్షరాలు	buffer సగటు	buffer గరిష్ఠం
200/నిమి	3.3	0.07	4
400/నిమి	6.7	0.60	8
800/నిమి	13.3	100.95	242
1600/నిమి	26.7	497.85	1069

ఒకే నియమం, రెండు పట్టికలు

టైపింగ్ వేగం > 1 ÷ round-trip → buffer అనంతంగా పెరుగుతుంది.

100 ms RTT = సెకనుకి 10 ops. 800/నిమిషం = సెకనుకి 13.3. 13.3 > 10 → buffer 100 కి చేరింది.

దీనికి నిజమైన పరిష్కారం: buffer lo ఉన్న ops ని **ఒకే సందేశంగా కలిపి** పంపడం. అప్పుడు పరిమితి "సెకనుకి ఎన్ని ops" కాదు, "సెకనుకి ఎన్ని సందేశాలు" అవుతుంది. **నేను దాన్ని ఈ doc lo అమలు చేయలేదు** – కాబట్టి దానికి ఒక సంఖ్య చెప్పడం లేదు. §16 lo దాని గురించి.

Part 4 – మూడో విరుపు: చరిత్ర

10. Step – server log ని ఉంచాలి

Server కి log ఎందుకు కావాలి? ఎందుకంటే ఒక client `baseRev = 500` తో op పంపినప్పుడు, server ప్రస్తుతం rev 520 దగ్గర ఉండొచ్చు. ఆ 20 ops కి వ్యతిరేకంగా transform చేయాలంటే – **ఆ 20 ops అక్కడ ఉండాలి.**

```
class Server {
  doc = []; log = []; #base = 0;
  get rev() { return this.#base + this.log.length; }

  receive(op, baseRev) {
    if (baseRev < this.#base) return { error: 'too-old', oldest: this.#base }; //
    §12
    let o = op;
    for (let i = baseRev - this.#base; i < this.log.length; i++) o = xf(o,
    this.log[i]);
    if (o.type !== 'nop') apply(this.doc, o);
    this.log.push(o);
    return { op: o, rev: this.rev };
  }
}
```

Log మూడు పనులు చేస్తుంది: **transform కి ఆధారం**, తిరిగి కలిసిన clients కి **catch-up**, మరియు వాడుకరికి కనిపించే **version history**.

కాబట్టి దాన్ని ఉంచాలి. ఎంత పెద్దది అవుతుంది?

11. మూడో విరుపు – log, document కంటే 134 రెట్లు

ఒక చురుకైన document – వారానికి 3 మంది, రోజుకి 90 నిమిషాలు, 35% సవరణలు/తొలగింపులు:

ఒక document · వారానికి 3 మంది × రోజుకి 90 నిమిషాలు

కనిపించే అక్షరాలు : 1,12,725 → 110.1 KB

log lo ops : 3,77,697 → 14.4 MB

log : document = 134×

110 KB DOCUMENT · 14.4 MB చరిత్ర

కారణం tombstones కాదు, తొలగింపులు కాదు. కారణం సులభమైనది: ఒక అక్షరం 1 byte, ఒక op 40 bytes – ఎందుకంటే op తో పాటు స్థానం, రకం, ఎవరు, ఏ revision అన్నీ రావాలి.

మరియు మీరు ప్రతి op ని ఉంచుతారు – అక్షరం తర్వాత తొలగించబడినా.

ఆ 40 bytes ఒక అంచనా, కాబట్టి దాని సున్నితత్వం:

ఒక్క op కి bytes	log పరిమాణం	document కి ఎన్ని రెట్లు
12	4.3 MB	40×
40	14.4 MB	134×
80	28.8 MB	268×

అత్యంత గట్టిగా binary-packed చేసినా 40 రెట్లు. ఇది encoding సమస్య కాదు.

ఇప్పుడు దాన్ని మొత్తం system కి పెంచితే:

ఇది ఒక doc. 10 బిలియన్ docs లో 1% ఇలా చురుకైనవి అయితే:

చురుకైన docs : 100 మిలియన్
వాటి text : 11.3 TB
వాటి op logs : 1.5 PB ← వారానికి

11.3 TB TEXT · వారానికి 1.5 PB చరిత్ర

వాడుకరులు రాసిన మొత్తం text ఒక మోస్తరు cluster లో పడుతుంది. వాళ్ళు ఎలా రాశారో అనే సమాచారం వారానికి అంతకంటే 130 రెట్లు.

సంవత్సరానికి **78 PB**, మరియు అది ఒక్క సంవత్సరం.

12. Step – snapshot + కత్తిరించిన log · offline కిటికీ

పరిష్కారం స్పష్టం: **document యొక్క snapshot ఉంచి, log ని కత్తిరించడం.**

```
truncate(keep) {  
  const drop = Math.max(0, this.log.length - keep);  
  this.log.splice(0, drop); this.#base += drop;  
  return drop;  
}
```

కానీ ఇక్కడే ఒక ఉచ్చు ఉంది, మరియు \$10 లో ఆ ఒక్క పంక్తి దాన్ని ముందే సూచించింది:

```
if (baseRev < this.#base) return { error: 'too-old', oldest: this.#base };
```

LOG ని కత్తిరించడం ఒక నిల్వ నిర్ణయం కాదు – అది ఒక **PRODUCT** నిర్ణయం

ఒక client offline lo edit చేసి, తర్వాత తిరిగి వస్తుంది. దాని `baseRev` ఎంత పాతదైనా server ఆ revision నుంచి transform చేయగలగాలి.

Log ని కత్తిరిస్తే, ఆ revision కంటే పాత clients కి **too-old** వస్తుంది – అంటే వాళ్ళ offline పని **స్వయంచాలకంగా కలపలేము**.

మీ log పొడవే మీ గరిష్ట offline కిటికీ.

ఆ కిటికీని కొలుద్దాం – ఈ doc lo గంటకి 35,971 ops:

log lo ఉంచేవి | నిల్వ | client offline ఉండగలిగే గరిష్ట సమయం

1,000		0.0 MB		2 నిమిషాలు
10,000		0.4 MB		17 నిమిషాలు
1,00,000		3.8 MB		2.8 గంటలు
10,00,000		38.1 MB		27.8 గంటలు

ఈ పట్టిక ఒక **ENGINEERING** నిర్ణయాన్ని ఒక **PRODUCT** హామీగా అనువదిస్తుంది

"వాడుకరులు ఒక రోజంతా offline lo పనిచేయగలగాలి" అంటే – ఒక్కే చురుకైన doc కి **38 MB log**.

10 కోట్ల చురుకైన docs × 38 MB = **3.8 PB**, కేవలం ఆ ఒక్క హామీ కోసం.

మరియు అది ఒక ఏకరీతి నిర్ణయం కానవసరం లేదు: **నెమ్మదైన doc lo గంటకి 100 ops మాత్రమే ఉంటే, అదే 1,00,000 ops log 41 రోజుల కిటికీ ఇస్తుంది.** కాబట్టి సరైన నియమం "N ops ఉంచు" కాదు – "T సమయం విలువైన ops ఉంచు".

మరియు **VERSION HISTORY**?

వాడుకరికి కనిపించే "ఈ doc చరిత్ర" అనేది **వేరే విషయం**, మరియు దానికి per-keystroke ops అవసరం లేదు.

దానికి కావాల్సింది అప్పుడప్పుడూ ఒక snapshot మరియు "ఎవరు ఎప్పుడు ఏ భాగాన్ని మార్చారు" అనే సారాంశం – **1.5 PB కాదు**.

కాబట్టి రెండు వేర్వేరు నిల్వలు: **వేడి** op log (transform కోసం, కత్తిరించబడినది) మరియు **చల్లని** చరిత్ర (snapshots, శాశ్వతం). వాటిని ఒకే దాంట్లో కలిపితే – మీరు transform కోసం PB లు ఉంచుతున్నారు.

Part 5 – నిరూపణ మరియు ప్రదర్శన

13. మొత్తం code • 1 లక్ష ops • mutation testing

500 యాదృచ్ఛిక sessions – 2 నుంచి 5 వాడుకరులు, 20 నుంచి 420 ms ఆలస్యం, చేర్పులు మరియు తొలగింపులు కలిపి. అయిదు నియమాలు:

1. **అభిసరణ (convergence)** – ప్రతి client యొక్క document server దానితో సరిగ్గా సమానం
2. **Log నిజాయితీ** – log ని మొదటి నుంచి replay చేస్తే అదే document రావాలి
3. **నకిలీలు లేవు** – ఒక అక్షరం రెండుసార్లు ఉండకూడదు
4. **ఏమీ మిగలలేదు** – ఏ client దగ్గరా pending ops ఉండకూడదు
5. **లెక్క సరిపోతుంది** – ins – del = పొడవు

500 యాదృచ్ఛిక sessions • 1,03,567 ops • 39,087 అక్షరాలు మిగిలాయి
నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

ins	68,752
del	34,815
nop	5,150
remote	2,92,902
acks	1,03,567
buffered	85,681
maxBuf	60

ఆ buffered 85,681 ఒక ముఖ్యమైన సంఖ్య: **1,03,567 ops lo 83% వెంటనే పంపబడలేదు** – అవి 99 యొక్క buffer lo ఆగాయి. అంటే ఆ దారి నిజంగా పరీక్షించబడింది, ఊహ కాదు.

"0 ఉల్లంఘనలు" అనేది ఒక **పరీక్ష విఫలం కాగలదని** నిరూపించాకే అర్థవంతమైనది:

==== mutation testing ====

మార్పు లేని code	→	0/500 విఫలం
ins/ins tie-break తీసేస్తే (ఎప్పుడూ a)	→	191/200 విఫలం
ins/del lo <= ని < గా మారిస్తే	→	192/200 విఫలం
del/ins lo < ని <= గా మారిస్తే	→	184/200 విఫలం
del/del ఒకే స్థానం → nop తీసేస్తే	→	68/200 విఫలం
ఉదా: లెక్క తప్పు: 107 ins – 75 del ≠ 34		
server transform తీసేస్తే	→	200/200 విఫలం
client తన in-flight op ని సర్దుబాటు చేయకపోతే	→	126/200 విఫలం
client వచ్చిన op ని in-flight కి సర్దుబాటు చేయకపోతే	→	200/200 విఫలం
buffer ని transform చేయకపోతే	→	198/200 విఫలం
ఒకేసారి ఒక op అనే నియమం తీసేస్తే (అన్నీ వెంటనే పంపితే)	→	200/200 విఫలం

తొమ్మిది మార్పులు, తొమ్మిదే పట్టుబడ్డాయి

చివరి వరుస ముఖ్యమైనది: "ఒకేసారి ఒక op" అనే నియమాన్ని తీసేస్తే 200 lo 200 విఫలం. అంటే 99 lo నేను చెప్పిన design నియమం ఒక అభిప్రాయం కాదు – అది కొలవబడినది.

మరియు del/del → nop కేవలం 68/200 lo విరిగింది. అది ఒక బలహీనమైన మార్పు కాదు – అది ఒక అరుదైన మార్పు. ఇద్దరు ఒకే అక్షరాన్ని ఒకే క్షణంలో తొలగించడం అరుదు, కాబట్టి ఆ bug production lo నెలల తర్వాత కనిపిస్తుంది.

Mutation testing ఒక చచ్చిన పంక్తిని కూడా కనుగొంది. నా మొదటి server lo ఒక షరతు ఉండేది – "ఈ op ఈ client దే అయితే transform చేయవద్దు". దాన్ని తీసేస్తే 0/200 విఫలం. కాబట్టి దాన్ని లెక్కపెట్టాను:

```
server lo "తన సొంత op" షరతు ఎన్నిసార్లు నిజమైంది: 0 / 103567 ops
```

ఒక్కసారి కూడా కాదు – ఎందుకంటే 99 యొక్క నియమం ప్రకారం client తన op ఆమోదించబడ్డాకే తర్వాతిది పంపుతుంది, కాబట్టి log [baseRev..] lo ఎప్పుడూ దాని సొంత op ఉండదు. ఆ పంక్తిని తీసేశాను.

ఇది ఒక నమూనా, మరియు ఇది ఈ SERIES LO నాలుగోసారి

Mutation testing యొక్క నిజమైన విలువ "నా పరీక్షలు మంచివా?" అని తెలుసుకోవడం మాత్రమే కాదు.

అది "ఈ code నిజంగా అవసరమా?" అని కూడా అడుగుతుంది. ఒక పంక్తిని తీసేసినా ఏమీ విరగకపోతే – రెండు అవకాశాలు: మీ పరీక్షలు బలహీనమైనవి, లేదా ఆ పంక్తి చచ్చినది.

రెండేది అయితే, దాన్ని ఉంచడం ఒక అబద్ధం – అది "ఈ సందర్భం జరుగుతుంది" అని తర్వాత చదివేవాళ్ళకి చెబుతుంది.

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

నిమిషాలు	ఏమి చెప్పాలి	ఏ సంఖ్య చెప్పాలి
0-5	Requirements + స్కోప్	10 బి docs · ఒక doc lo 50 మంది
5-9	Docs మధ్య shard సులభం, doc లోపల అసాధ్యం	సెకనుకి 8,167 సందేశాలు ఒక doc కి
9-14	పూర్తి doc save → LWW	నష్టం (N-1)/N · తరచుగా save చేస్తే మారదు
14-20	ops పంపడం → స్థానాలు విరుగుతాయి	0/60 sessions · 0 ms తోనూ
20-30	Transform – అరు సందర్భాలు, మూడు సూక్ష్మమైనవి	tie-break తీస్తే 191/200
30-35	ఒకేసారి ఒక op · buffer గోడ	టైపింగ్ వేగం > 1÷RTT → అనంతం
35-42	Log 134 రెట్లు · snapshot · offline కిటికీ	1.5 PB/వారం · 38 MB = 1 రోజు
42-45	Follow-ups – cursors, undo, rich text	\$16

ఈ PROBLEM LO మిమ్మల్ని వేరుగా నిలబెట్టే ఒక్క క్షణం

చాలామంది "OT వాడతాను" లేదా "CRDT వాడతాను" అని పేరు చెప్పి ముందుకి వెళ్తారు.

మీరు బదులుగా ఒక ఉదాహరణ చెప్పండి: "A B C D E – ఒకరు స్థానం 1 lo X, ఇంకొకరు స్థానం 3 lo Y. ఆ రెండు ops మార్చుకున్నాక ఒకరికి A X B Y C D E, ఇంకొకరికి A X B C Y D E. 'స్థానం 3' అనేది ఒక సత్యం కాదు, ఒక అభిప్రాయం – ఏ స్థితి నుంచి చూస్తున్నామో దాని మీద ఆధారపడుతుంది."

ఆ ఒక్క ఉదాహరణ చెబితే మీకు **transform ఎందుకు ఉండో** తెలుసని స్పష్టమవుతుంది – దాని పేరు తెలుసని కాదు.

15. నేటితో చెప్పాల్సిన English script

Sharding గురించి:

"The first thing I'd say is that this problem is unusual: sharding across documents is trivial, and sharding within a document is impossible. Someone has to decide whose character goes where, and that ordering can't be split across machines. So one document is one process. The good news is the load per document is small – fifty simultaneous editors is about eight thousand messages a second. Everything interesting happens inside that one process."

LWW గురించి:

"If I save the whole document periodically with last-write-wins, I measured the loss as exactly $N - 1$ over N – with ten editors, ninety percent of the typing is gone. And saving more often doesn't help at all: I ran it from five seconds down to a hundred milliseconds and the loss stayed at sixty-seven percent. Frequent saves make the loss happen sooner, not less. The fix isn't a faster save – it's sending the *intent* instead of the *result*."

Divergence గురించి:

"So I send operations – insert at position five. I ran sixty sessions and zero of them converged; with two hundred milliseconds of latency, fifty-nine percent of the final document differed between replicas. And it still diverged with zero latency, which is the point: this isn't a network problem. Two people typing at the same moment is concurrency no matter how fast the wire is. Position three means 'after C' to the sender, but by the time that message lands, position three is somewhere else."

Transform గురించి:

"That's what transform does – it translates an operation from the state it was written against into the state it's being applied to. Six cases for insert and delete. Three of them are subtle: the tie-break when two people insert at exactly the same spot, the asymmetry between insert-versus-delete and delete-versus-insert, and two deletes of the same character becoming a no-op. I mutation-tested each one; removing the tie-break breaks a hundred ninety-one of two hundred sessions."

History గురించి:

"The last thing is storage, and it surprised me. For an actively co-authored document the op log came out a hundred thirty-four times the size of the document itself – not because of tombstones, but because a character is one byte and an operation is forty. Even packed to twelve bytes it's forty times. Across a hundred million active documents that's about a petabyte and a half a week. So you snapshot and truncate – and the moment you truncate, the length of your log is your maximum offline window. Keeping a million operations is thirty-eight megabytes and buys you about a day."

16. Follow-ups – cursors, undo, offline, CRDT

"ఇతరుల cursor ఎక్కడ ఉండే ఎలా చూపిస్తారు?"

Cursor ఒక స్థానం – కాబట్టి దానికీ అదే transform వర్తిస్తుంది. ఒక remote op వచ్చినప్పుడు, మీ దగ్గర ఉన్న ప్రతి ఇతర వాడుకరి cursor ని ఆ op కి వ్యతిరేకంగా transform చేయాలి (ins లాగా, కానీ ఏమీ చేర్చకుండా). ఇది చేయకపోతే cursors ట్రైపింగ్ తో నెమ్మదిగా జారిపోతాయి. Cursors నిల్వ చేయాల్సిన అవసరం లేదు – అవి క్షణికమైనవి, మరియు ఒక వాడుకరి వెళ్ళిపోతే అవి పోవాలి.

"Undo ఎలా?"

ఇది అనుకున్నదానికంటే చాలా కష్టం. Ctrl+Z అంటే "నా చివరి మార్పుని రద్దు చెయ్యి" – "documentకి జరిగిన చివరి మార్పు" కాదు. కాబట్టి మీ పాత op ని తిరగేసి (ins → del), ఆ తర్వాత జరిగిన అన్ని ops కి వ్యతిరేకంగా transform చేసి, ఒక కొత్త op గా పంపాలి. మీరు రద్దు చేస్తున్న అక్షరాన్ని ఇంకొకరు ఇప్పటికే తొలగించి ఉంటే అది nop అవుతుంది. ఇది ఒక కొత్త op, ఒక వెనక్కి వెళ్ళడం కాదు – history ఎప్పుడూ ముందుకే సాగుతుంది.

"OT కి బదులు CRDT ఎందుకు కాదు?"

CRDT ప్రతి అక్షరానికి ఒక శాశ్వత ప్రత్యేక గుర్తింపు ఇస్తుంది, కాబట్టి స్థానాలు ఎప్పుడూ మారవు మరియు transform అవసరం లేదు. దీని పెద్ద లాభం: కేంద్ర server అవసరం లేదు, కాబట్టి peer-to-peer మరియు దీర్ఘకాల offline పనిచేస్తాయి.

ఖర్చు: ప్రతి అక్షరం ఒక id మోయాలి, మరియు తొలగించిన అక్షరాలు tombstones గా మిగలాలి (ఇతరుల ops వాటిని సూచించవచ్చు). §11 lo నేను కొలిచింది OT యొక్క op log – CRDT lo ఆ భారం document లోపలికే

వెళ్తుంది. నేను CRDT ని అమలు చేయలేదు, కాబట్టి ఏది చిన్నదో నేను చెప్పను. నిజాయితీగా చెప్పాల్సింది: రెండింటికీ చరిత్ర ఖర్చు ఉంది, మరియు OT lo అది సర్వర్ మీద మరియు కత్తిరించదగినది, CRDT lo అది ప్రతి replica మీద.

"§9 యొక్క buffer గోడని ఎలా పరిష్కరిస్తారు?"

Buffer lo ఉన్న ops ని ఒకే సందేశంగా పంపడం. Server వాటిని వరుసగా వర్తింపజేయాలి, మరియు ప్రతి దానికి ఏకకాల ops ని ముందుకి transform చేస్తూ వెళ్ళాలి – ఒక జాబితా-వర్సెస్-జాబితా transform. నేను దీన్ని అమలు చేయలేదు, కాబట్టి దీని గురించి నాకు కొలిచిన సంఖ్య లేదు. కానీ §9 యొక్క రెండు పట్టికలు ఇది ఎప్పుడు అవసరమో ఖచ్చితంగా చెబుతాయి: టైపింగ్ వేగం round-trip యొక్క విలోమాన్ని దాటినప్పుడు.

"Rich text – bold, పట్టికలు, బొమ్మలు?"

Transform సందర్భాలు ఆరు నుంచి పెరుగుతాయి, ఎందుకంటే ఇప్పుడు `format(range, attr)` అనే మూడో రకం op ఉంది, మరియు అది `ins / del` రెండింటితో కలవాలి. పట్టికలు ఇంకా ఫూరం – ఒక పట్టిక ఒక వ్యక్షం, మరియు వ్యక్షాల మీద transform (ఒకరు ఒక node ని తరలిస్తుంటే ఇంకొకరు దాని parent ని తొలగిస్తే) పరిష్కరించని పరిశోధన సమస్యలను కలిగి ఉంది. ఆచరణలో: పత్రాన్ని సరళక్రమంగా (flat sequence) సూచించి, నిర్మాణాన్ని ప్రత్యేక గుర్తులుగా encode చేయడం.

17. ఏమి నేర్చుకున్నాం

1. స్థితిని పంచకండి, మార్పులను పంచండి. పూర్తి document ని పంపడం ఒక ఫలితాన్ని పంపడం – దాన్ని కలపలేరు. నష్టం సరిగ్గా $(N-1)/N$.
2. ఒక పరిష్కారాన్ని వేగవంతం చేయడం దాన్ని సరైనదిగా చేయదు. Save విరామం 5 సెకన్ల నుంచి 100 ms కి – నష్టం ఒక్క దశాంశం కూడా మారలేదు.
3. స్థానం ఒక సత్యం కాదు, ఒక అభిప్రాయం. "స్థానం 3" అనేది ఏ స్థితి నుంచి చూస్తున్నామో దాని మీద ఆధారపడుతుంది. అందుకే transform ఉంది.
4. ఇది network సమస్య కాదు. 0 ms ఆలస్యంతోనూ 0/60 sessions ఒకేలా ముగిశాయి. ఏకకాలాన్ని సృష్టించేది network కాదు – ఇద్దరు మనుషులు.
5. అరుదైనది అంటే సురక్షితమైనది కాదు. `del/del → nop` ని తీసేస్తే 68/200 మాత్రమే విరిగాయి – మిగతా వాటికంటే తక్కువ, ఎందుకంటే ఆ పరిస్థితి అరుదు. అది ఆ bug ని ఆలస్యంగా బయటపడేది చేస్తుంది, తక్కువ తీవ్రమైనది కాదు.
6. ఒక design నియమాన్ని కూడా కొలవొచ్చు. "ఒకేసారి ఒక్క op" అనేది నేను చదివిన నియమం కాదు – నా code 500/500 విఫలమయ్యాక కనుగొన్నది, మరియు దాన్ని తీసేస్తే 200/200 విరుగుతాయి.
7. ఒక పరిమితిని రెండు వైపుల నుంచి కొలవాలి. Buffer గోడ ఆలస్యం పెంచినా, టైపింగ్ వేగం పెంచినా వస్తుంది – ఎందుకంటే నియమం టైపింగ్ వేగం vs $1+RTT$.
8. చరిత్ర ఒక నిల్వ సమస్య కాదు – ఒక product నిర్ణయం. Log document కంటే 134 రెట్లు, వారానికి 1.5 PB. దాన్ని కత్తిరించిన క్షణంలో మీరు "వాడుకరి ఎంతసేపు offline ఉండొచ్చు" అని నిర్ణయించారు.

9. Mutation testing చచ్చిన code ని కూడా కనుగొంటుంది. ఒక పంక్తి తీసేస్తే **0/200** విఫలం – లెక్కపెట్టి చూస్తే ఆ షరతు **1,03,567 ops lo 0 సార్లు** నిజమైంది.

ఈ **DOC** నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

"వెంటనే కనిపించాలి" అనే ఒక్క అవసరం ఈ problem మొత్తాన్ని సృష్టిస్తుంది.

Server ఆమోదం వరకు ఆగితే ఒక queue సరిపోతుంది – transform లేదు, buffer లేదు, divergence లేదు.

"వెంటనే" అంటే client server ని నమ్మకుండా ముందుకి వెళ్తుంది. ఆ క్షణంలో మీ దగ్గర **ఒక document కాదు, N వేర్వేరు documents** ఉన్నాయి – మరియు మిగతా మొత్తం design వాటిని తిరిగి ఒకటిగా చేయడం గురించి.

ప్రతి "instant" UI వెనక ఇదే ఒప్పందం ఉంది. వాడుకరికి కనిపించే వేగం, engineer కి వచ్చే సంక్లిష్టత.